

Informe de Práctica: Redes Neuronales Profundas

Nombre: Darel Martínez Caballero

Curso académico: 2025/2026

Asignatura: PIA (Programación de Inteligencia Artificial)

Ejercicio: PIA05

Índice

1. Objetivos del documento

2. Desarrollo

Ejercicio 1: Importación de Módulos

Ejercicio 2: Carga del Dataset

Ejercicio 3: Exploración y Preprocesamiento

Ejercicio 4: Creación del Modelo Base

Ejercicio 5: Entrenamiento del Modelo Base

Ejercicio 6: Mejora del Modelo

Ejercicio 7: Evaluación de Resultados

Ejercicio 8: Visualización de Predicciones

3. Conclusiones finales

4. Bibliografía

1. Objetivos del documento

El presente documento tiene como objetivo documentar detalladamente el desarrollo de la práctica PIA05, centrada en la implementación, entrenamiento y evaluación de modelos de Redes Neuronales Convolucionales (CNN) para la clasificación de imágenes utilizando el dataset CIFAR-10.

2. Desarrollo

Ejercicio 1: Importación de Módulos

Se importan las librerías fundamentales para el desarrollo de redes neuronales con Keras (backend de TensorFlow), así como NumPy para manipulación numérica y Matplotlib para visualización.

```
import numpy as np
from keras.models import Sequential
from keras.layers import Flatten, Dense, Input, Conv2D, MaxPooling2D
from keras.datasets import cifar10
from keras.utils import to_categorical
from matplotlib import pyplot as plt
```

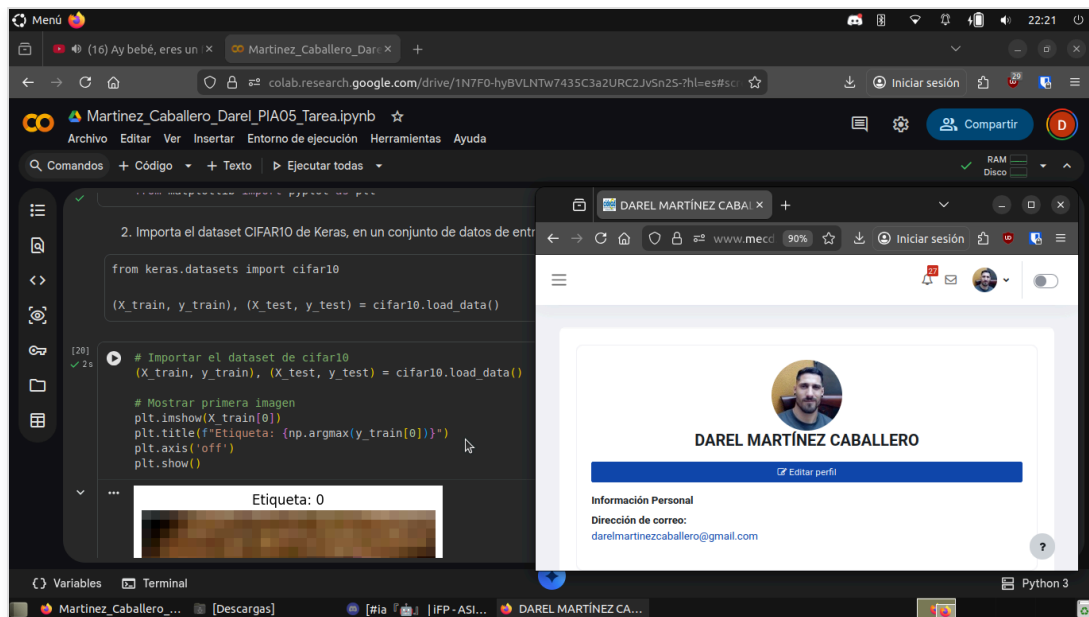


Figura 1. Importación de módulos y configuración inicial.

Ejercicio 2: Carga del Dataset

Se carga el dataset CIFAR-10, dividiéndolo en conjuntos de entrenamiento (train) y prueba (test). Se visualiza la primera imagen del conjunto de entrenamiento para verificar la carga correcta.

```
(X_train, y_train), (X_test, y_test) = cifar10.load_data()
plt.imshow(X_train[0])
```

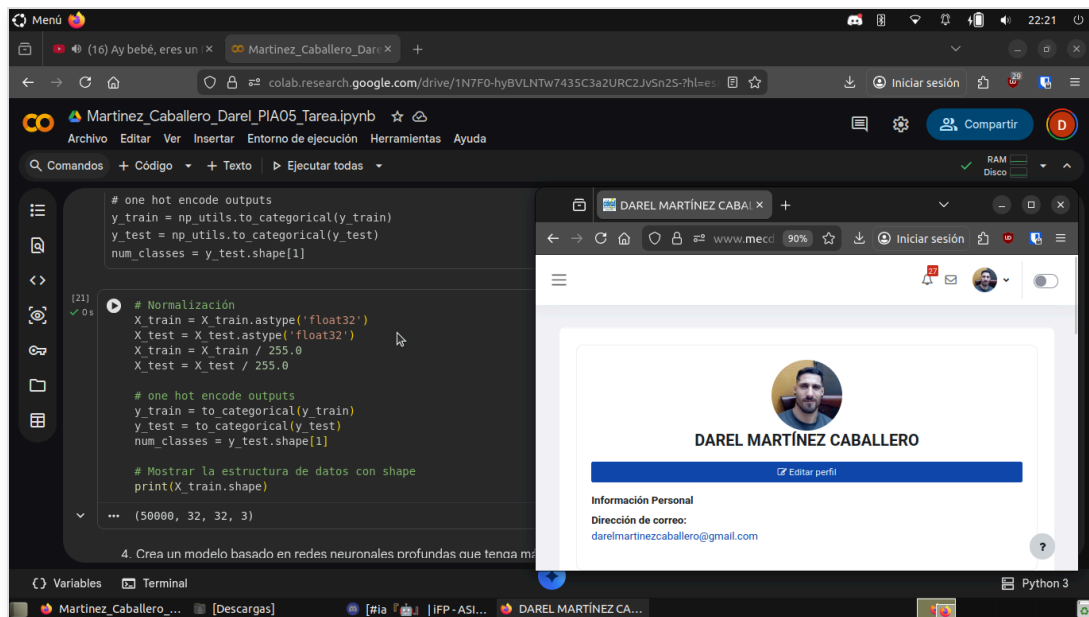


Figura 2. Carga y visualización inicial del dataset.

Ejercicio 3: Exploración y Preprocesamiento

Se normalizan los datos de entrada dividiendo los valores de píxeles por 255.0 para obtener un rango entre 0 y 1. Las etiquetas se convierten a formato *one-hot encoding* utilizando `to_categorical`.

```
X_train = X_train.astype('float32') / 255.0
X_test = X_test.astype('float32') / 255.0
y_train = to_categorical(y_train)
y_test = to_categorical(y_test)
# Dimensiones: (50000, 32, 32, 3)
```

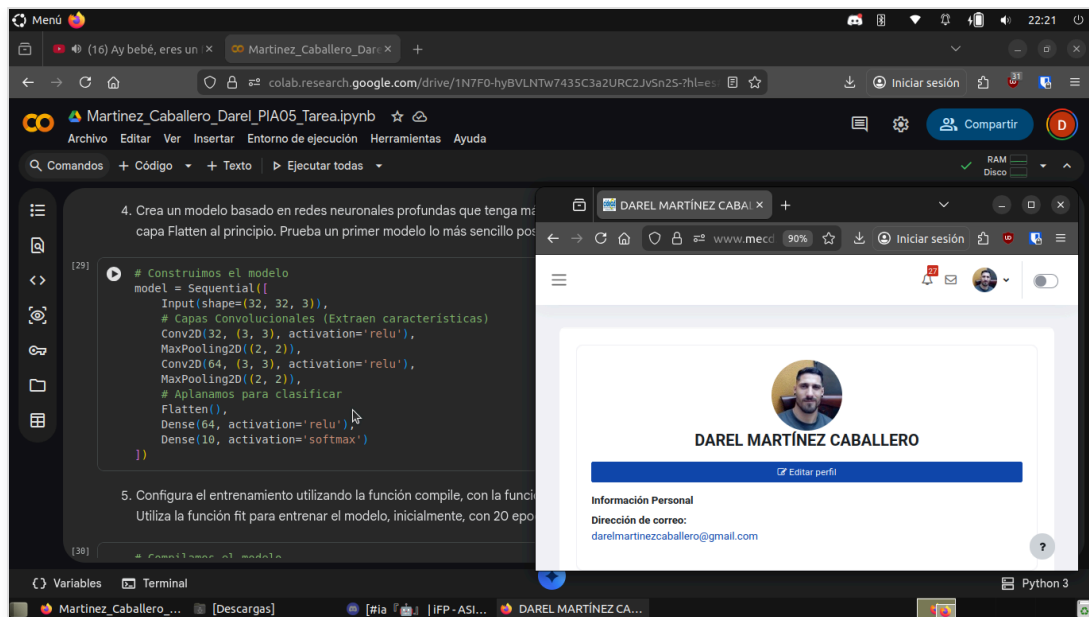


Figura 3. Normalización y verificación de dimensiones.

Ejercicio 4: Creación del Modelo Base

Se construye un modelo secuencial básico con capas convolucionales (Conv2D), de pooling (MaxPooling2D), aplanado (Flatten) y capas densas (Dense).

```
model = Sequential([
    Input(shape=(32, 32, 3)),
    Conv2D(32, (3, 3), activation='relu'),
    MaxPooling2D((2, 2)),
    Conv2D(64, (3, 3), activation='relu'),
    MaxPooling2D((2, 2)),
    Flatten(),
    Dense(64, activation='relu'),
    Dense(10, activation='softmax')
])
```

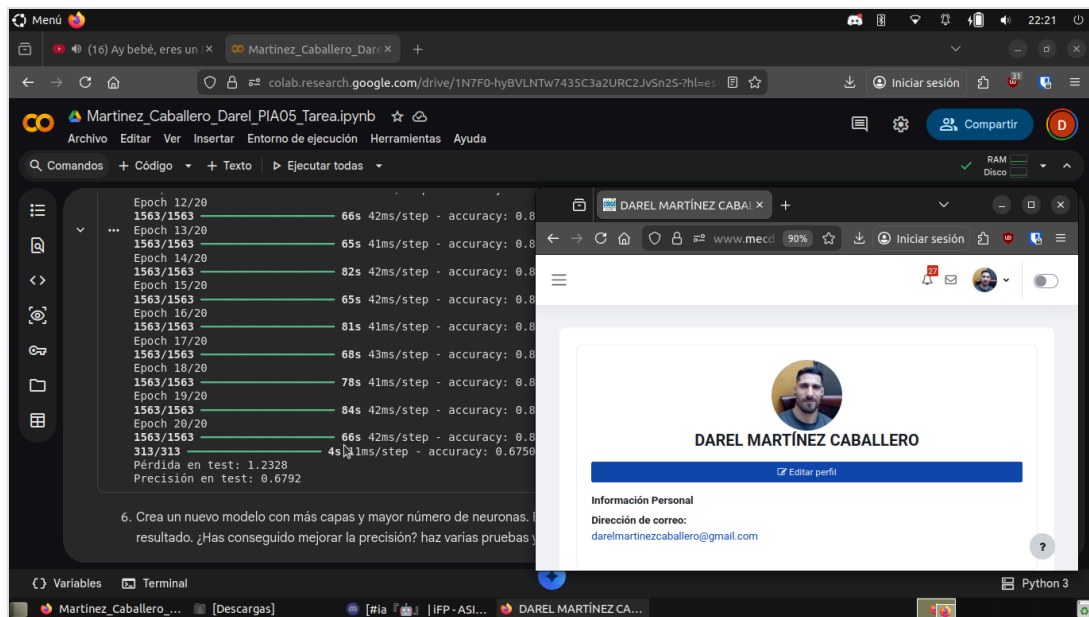


Figura 4. Definición de la arquitectura del primer modelo.

Ejercicio 5: Entrenamiento del Modelo Base

Se compila el modelo con el optimizador Adam y función de pérdida de entropía cruzada categórica. Se entrena durante 20 épocas.

Resultados obtenidos:

Precisión final en entrenamiento (Epoch 20): ~87.5%

Precisión en test: 67.92%

Pérdida en test: 1.2328

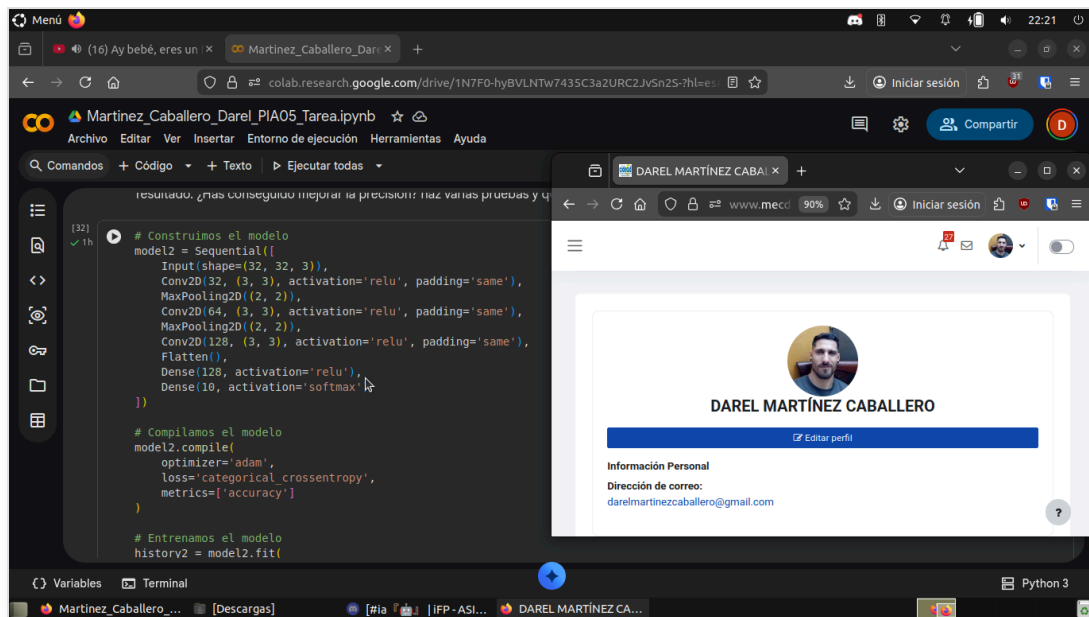


Figura 5. Proceso de entrenamiento y métricas del primer modelo.

Ejercicio 6: Mejora del Modelo

Se diseña una arquitectura más profunda (añadiendo una capa Conv2D de 128 filtros y capas densas más grandes) y se entrena durante 50 épocas para intentar mejorar el rendimiento.

```
model2 = Sequential([
    ...
    Conv2D(128, ...),
    ...
    epochs=50
    ...
])
```

Resultados Modelo Mejorado:

Precisión final en entrenamiento: 98.68%

Precisión en test: 69.77%

Pregunta: ¿Has conseguido mejorar la precisión?

Sí, se ha conseguido una mejora ligera en la precisión del conjunto de test, pasando de un 67.92% a un 69.77%. Sin embargo, la mejora en el conjunto de entrenamiento ha sido drástica (llegando casi al 99%), lo cual tiene implicaciones importantes sobre el ajuste del modelo que se analizan a continuación.

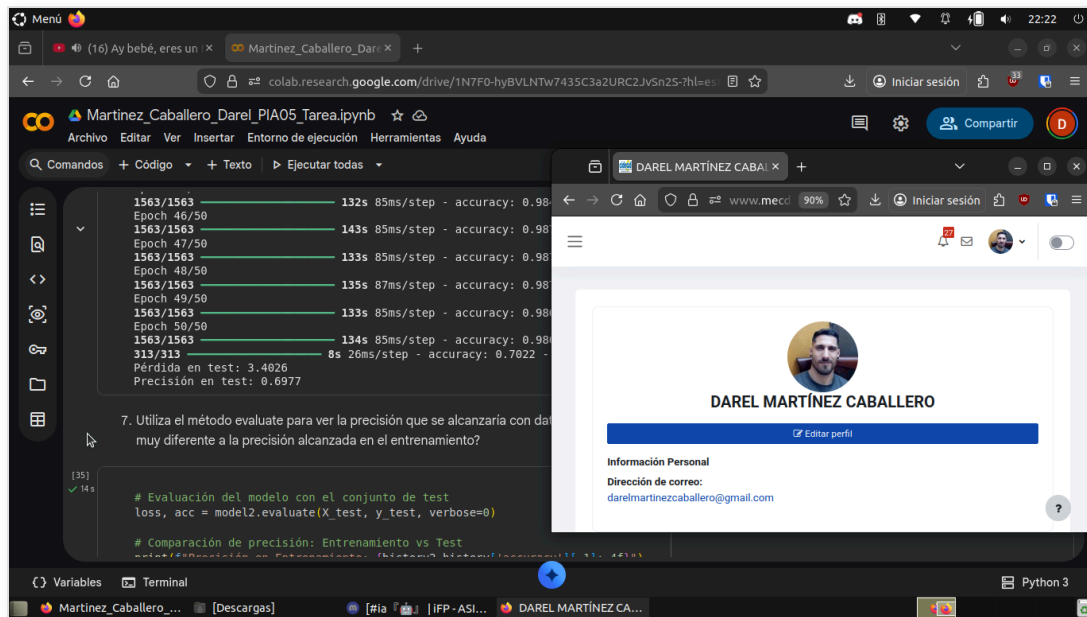


Figura 6. Entrenamiento del segundo modelo (50 epochs).

Ejercicio 7: Evaluación de Resultados

Se compara el rendimiento del modelo entre los datos de entrenamiento y los datos de validación/test.

Pregunta: ¿Es muy diferente a la precisión alcanzada en el entrenamiento?

Sí, existe una diferencia muy significativa. Mientras que en entrenamiento se alcanza un **98.46%** de precisión, en test se obtiene solo un **69.77%**. Una brecha de casi 30 puntos porcentuales indica un claro caso de **Overfitting (sobreajuste)**: el modelo ha "memorizado" los datos de entrenamiento pero no generaliza correctamente ante datos nuevos.

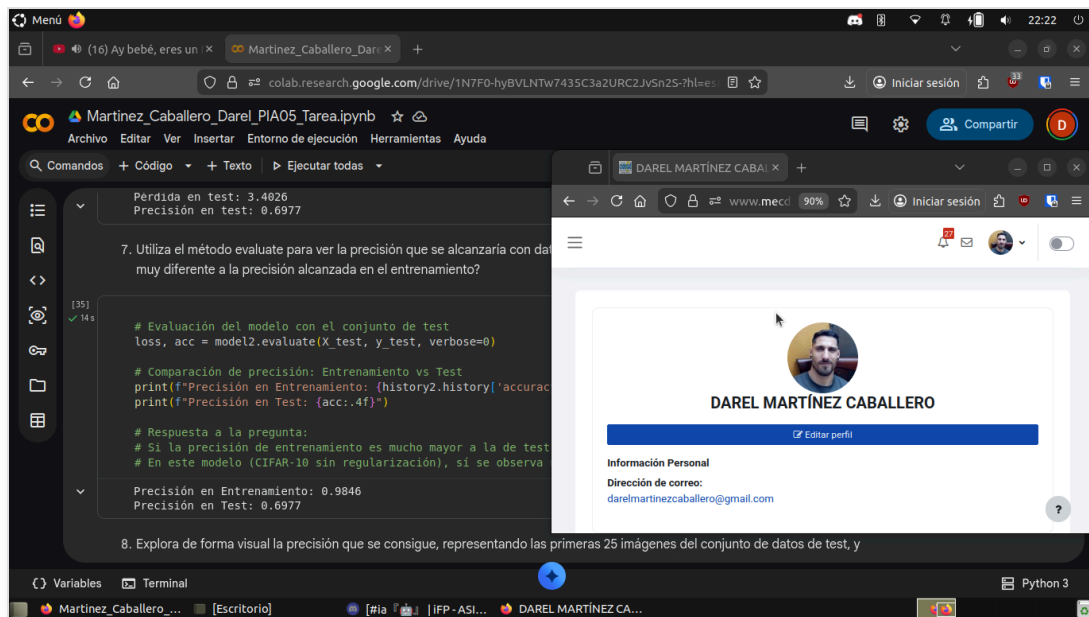


Figura 7. Comparativa de precisión (Train vs Test).

Ejercicio 8: Visualización de Predicciones

Se visualizan las predicciones del modelo sobre las primeras 25 imágenes del conjunto de test. Las etiquetas verdes indican que la predicción coincide con la etiqueta real (acierto), mientras que las rojas indican un error.

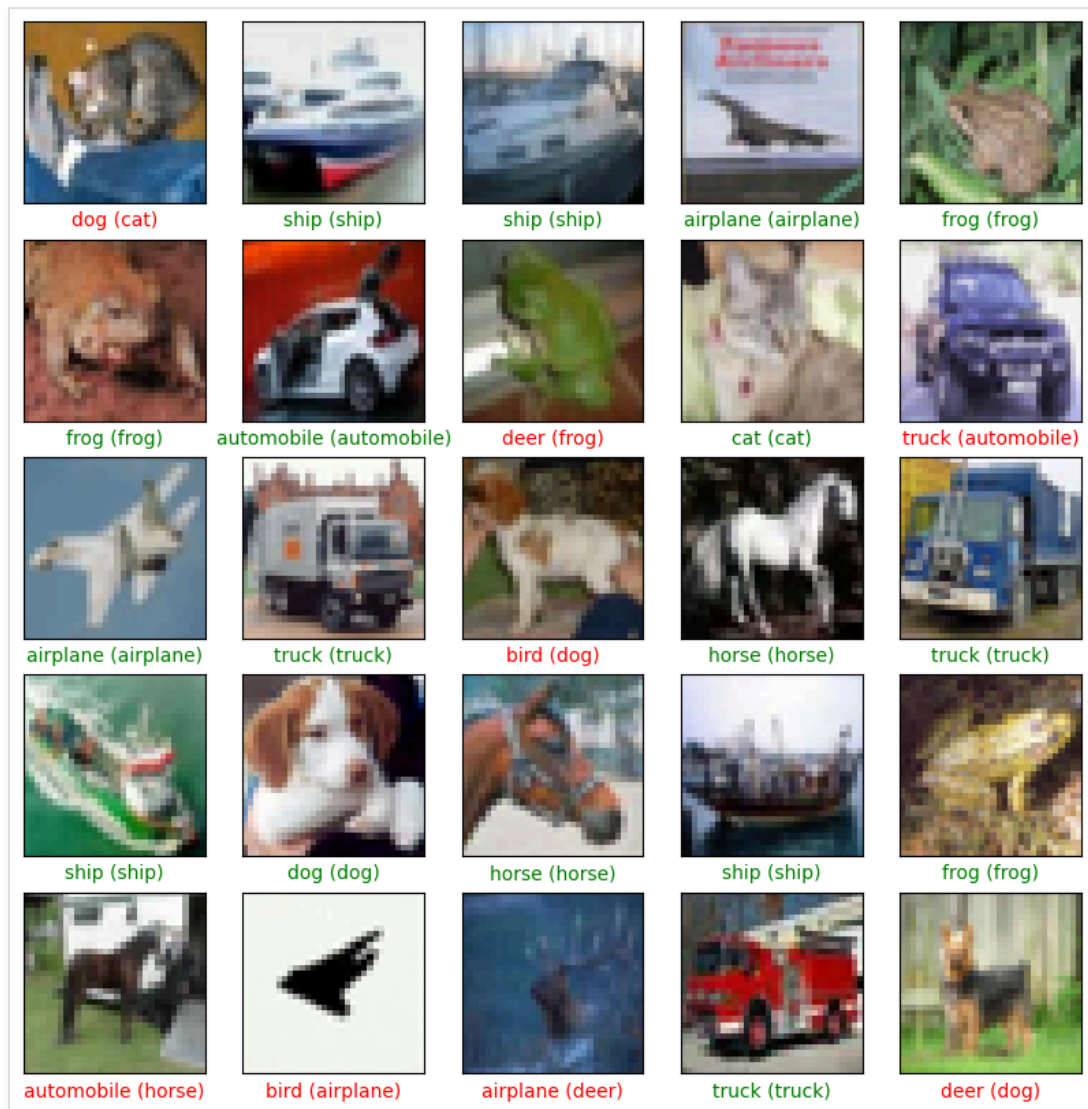


Figura 8. Matriz de 25 imágenes del conjunto de test con sus predicciones.

Análisis de la imagen:

En la Figura 8 se observa una muestra representativa del desempeño del modelo. Dada la precisión global del ~69%, es esperable observar varios errores (texto en rojo) en esta muestra. El modelo tiende a clasificar correctamente objetos con formas distintivas y fondos contrastados (como aviones o barcos en el cielo/mar), pero muestra confusión en categorías con características visuales compartidas, como puede ser la distinción entre animales cuadrúpedos (gatos, perros, ciervos) cuando la imagen es de baja resolución o tiene fondos complejos.

3. Conclusiones finales

Síntesis del trabajo:

En esta práctica se ha abordado la clasificación de imágenes del dataset CIFAR-10 mediante Redes Neuronales Convolucionales. Se comenzó con un modelo simple que alcanzó un 67% de precisión y se evolucionó a una arquitectura más compleja entrenada por más tiempo, logrando casi un 70% en test.

Análisis crítico:

El resultado más relevante es la detección de un severo **overfitting** en el segundo modelo. Aumentar la complejidad de la red y las épocas de entrenamiento mejoró drásticamente el aprendizaje de los datos de entrenamiento (casi 99%), pero apenas impactó en la validación. Esto demuestra que simplemente "hacer la red más grande" no es suficiente.

Aprendizajes:

Se ha consolidado el conocimiento sobre la estructura de las CNNs (capas Conv2D, Pooling), el preprocesamiento de imágenes y la interpretación de curvas de aprendizaje. Se ha evidenciado empíricamente cómo el desequilibrio entre la capacidad del modelo y la regularización conduce al sobreajuste.

Acceso al Notebook

Para consultar el código fuente y la ejecución en vivo, puede acceder al siguiente enlace de Google Colab:

[Ver Notebook en Google Colab](#)

4. Bibliografía

Keras Team. (2024). *Keras Documentation: CIFAR10 small images classification dataset*. Recuperado de <https://keras.io/api/datasets/cifar10/>